

# Exercícios — Aula 01 — Revisão, pseudocódigo, arrays e contagem de operações

Técnicas de Programação - 2026/02

Última atualização: September 2, 2026

## Instruções

- Leia o pseudocódigo antes de executar mentalmente.
- Para cada simulação, acompanhe índices, acumuladores, comparações e saída.
- Só escreva sua própria solução depois de conseguir prever o comportamento em entradas pequenas.

## Exercício 1 — Simulação de maior nota e aprovados

Simule o algoritmo abaixo para `notas = [7.5, 4.0, 9.0, 6.0, 5.5]`.

Preencha a tabela com os valores de `i`, `nota`, `maior`, `aprovados` e `comparacoes` ao final de cada repetição.

**Algorithm 1:** RelatorioNotas

**Result:** Executa RELATORIO-NOTAS.

**Input** : array notas

**Output:** maior nota, quantidade de aprovados e quantidade de comparações

```
if TAMANHO(notas) = 0 then
    return (-1, 0, 0)
end
maior ← notas[0]
aprovados ← 0
comparacoes ← 0
for i ← 0 to TAMANHO(notas) - 1 do
    comparacoes ← comparacoes + 1
    if notas[i] ≥ 6.0 then
        aprovados ← aprovados + 1
    end
    if i > 0 then
        comparacoes ← comparacoes + 1
        if notas[i] > maior then
            maior ← notas[i]
        end
    end
end
return (maior, aprovados, comparacoes)
```

| i | notas[i] | maior ao final | aprovados ao final | comparacoes ao final |
|---|----------|----------------|--------------------|----------------------|
| 0 |          |                |                    |                      |
| 1 |          |                |                    |                      |
| 2 |          |                |                    |                      |
| 3 |          |                |                    |                      |
| 4 |          |                |                    |                      |
| 5 |          |                |                    |                      |

Qual é a saída final?



### Exercício 2 — Divisibilidade

Simule o algoritmo abaixo para  $N = 7$ ,  $N = 22$ ,  $N = 150$  e  $N = 9$ .

**Algorithm 2:** Divisibilidade

**Result:** Executa DIVISIBILIDADE.

**Input :** int N

**Output:** string

```
if N mod 2 = 0 and N mod 3 ≠ 0 then
|   return "Ins"
end
if N mod 2 ≠ 0 and N mod 3 = 0 then
|   return "per"
end
if N mod 2 = 0 and N mod 3 = 0 then
|   return "Insper"
end
return ""
```

| N   | primeira condição | segunda condição | terceira condição | retorno |
|-----|-------------------|------------------|-------------------|---------|
| 7   |                   |                  |                   |         |
| 22  |                   |                  |                   |         |
| 150 |                   |                  |                   |         |
| 9   |                   |                  |                   |         |

Depois da simulação, explique por que a ordem das condições não muda o resultado neste algoritmo.

### Exercício 3 — Índice fora do array

O pseudocódigo abaixo tenta somar todos os elementos, mas contém um erro de índice.

---

**Algorithm 3:** SomaComErro

---

**Result:** Executa SOMA-COM-ERRO.**Input :** array  $v$ **Output:** number $soma \leftarrow 0$ **for**  $i \leftarrow 0$  **to**  $TAMANHO(v)$  **do**|  $soma \leftarrow soma + v[i]$ **end****return**  $soma$ 

---

1. Simule para  $v = [4, 8, 1]$  até o ponto em que o problema aparece.
2. Qual índice inválido é acessado?
3. Reescreva apenas a linha do **PARA** corrigida.

## Exercício 4 — Soma e média

Escreva pseudocódigo para o algoritmo `MEDIA-ARRAY`.

Contrato:

- entrada: array `v` de números;
- saída: média dos elementos;
- se `v` estiver vazio, retorne 0.

Antes de escrever a versão final, teste sua ideia com:

- `v = []`;
- `v = [10]`;
- `v = [4, 6, 8]`.

## Exercício 5 — Contagem de operações

Considere o algoritmo:

---

**Algorithm 4:** ContaPares

---

**Result:** Executa CONTA-PARES.

**Input :** array  $v$

**Output:** int

$pares \leftarrow 0$

**for**  $i \leftarrow 0$  **to**  $TAMANHO(v) - 1$  **do**

**if**  $v[i] \bmod 2 = 0$  **then**

$pares \leftarrow pares + 1$

**end**

**end**

**return**  $pares$

---

Para um array de tamanho  $n$ , responda:

1. Quantas vezes a condição  $v[i] \bmod 2 = 0$  é avaliada?
2. Essa quantidade depende dos valores dentro do array?
3. O custo cresce de forma constante, linear ou quadrática?

## Exercício 6 — Um laço ou dois laços

Os dois algoritmos abaixo trabalham com um array de tamanho  $n$ .

---

### Algorithm 5: A

---

**Result:** Executa A.

**Input :** array  $v$

**Output:** int

```
total  $\leftarrow$  0
for  $i \leftarrow 0$  to TAMANHO( $v$ ) - 1 do
    | total  $\leftarrow$  total +  $v[i]$ 
end
return total
```

---

---

### Algorithm 6: B

---

**Result:** Executa B.

**Input :** array  $v$

**Output:** boolean

```
for  $i \leftarrow 0$  to TAMANHO( $v$ ) - 1 do
    for  $j \leftarrow i + 1$  to TAMANHO( $v$ ) - 1 do
        | if  $v[i] = v[j]$  then
        |     | return true
        | end
    end
end
return false
```

---

1. Qual algoritmo faz uma quantidade de trabalho proporcional a  $n$ ?
2. Qual algoritmo compara pares de posições?
3. Para  $v = [1, 2, 3, 4]$ , quantas comparações  $v[i] = v[j]$  o Algoritmo B faz?
4. Explique por que o Algoritmo B cresce mais rápido quando o array aumenta.